ece560-hw3

Table of Contents

- [x] ~~Question 0: Accessing the Homework (0 points, but necessary)~~
- [x] ~~Question 1: Chapter 3 - User Authentication (7 points)~~
- [x] ~~Question 2: File permissions (7 points)~~
- [x] ~~Question 3: Data processing (16 points)~~
  - [x] ~~part 1: nmap~~
  - [x] ~~part 2: logwatch~~
  - [x] ~~part 3: website crawler~~
- [x] ~~Question 4: Regular expressions (9 points)~~
  - [x] ~~part 1: netids~~
  - [x] ~~part 2: urls~~
  - [x] ~~part 3: pi~~
- [x] ~~Question 5: Manual user management (6 points)~~
- [x] ~~Question 6: GNU Screen (2 points)~~
- [x] ~~Question 7: Cracking MD5 hashes with hashcat (5 points)~~
- [x] ~~Question 8: John the Ripper (6 points)~~
- [ ] Question 9: Evaluating MFA approaches (8 points)
- [x] ~~Question 10: SSH with MFA (5 points)~~
- [x] ~~Question 11: Hydra (Online SSH Dictionary Attack) (5 points)~~
- [x] ~~Question 12: Craft your prompt (3 points)~~
- [x] ~~Question 13: Attack recon (6 points)~~
- [ ] Question 14: Keeping up with the news (10 points)
- [ ] Question 15: Manipulating binary file formats (5 points)

Question 1: User Authentication (7 points)

Answer the questions below. (Based in part on review questions from the textbook)

1. ***In general terms, what are the three (or four) categories of ways to authenticate a user's identity?***
   Something you know (e.g. password)
   Something you have (e.g. token)
   Something you are (e.g. biometric)
   ~~Something you do (e.g. voice)~~ increasingly obsolete
2. ***List and briefly describe five possible threats to the secrecy of passwords.***

- Dictionary Attack (cracking a stolen hash, trying guesses from a dictionary on a live system)
- Password Spraying (trying common passwords on many accounts)
- Credential Stuffing (attackers using previously leaked passwords)
- Password Guessing (guess a user's password based off of research)
- Electronic Monitoring (keyloggers, sniffing the network)

1. ***What are two enhancements beyond basic hashing to protect stored passwords?***
   Salting: appending a random, known value onto the user's password before hashing
   Hash iterations: slowing down the process of hash calculation by running the hash function on the input password more times, making a brute force approach much more computationally intensive.
2. ***Which is more important: password complexity requirements or password expiration requirements? Why?***
   Password complexity requirements. Password expiration requirements are generally not very effective, as users are likely to use a password similar to their previous one when asked to create a new password, and attackers usually do not need long-term access to a system in order to accomplish their goals.
3. ***What is MFA? What threat model does it deal with?***
   Multi-factor authentication. This deals with the threat model of having one's credentials compromised. If the attacker has the capability to figure out the password of a user, they would be able to access the asset that is the user's account. If we use MFA, in order to log into the user's account, an attacker would also have to steal another credential belonging to the user, which would prove much more difficult. This essentially gives more redundancy to a user authentication system
4. ***Describe the general concept of a challenge-response protocol.***
   In order to defend against replay attacks, the guarding party sends a random value that they ask the person who wishes to log in to encrypt. This shows that the incoming party actually knows the credential and is not just replaying a previous login attempt.
5. ***In a challenge-response password protocol, why is a random nonce used?***
   This way, not only is the challenge different every single session, but there are also no patterns in the order of challenges/responses that attackers may be able to exploit.

Question 2: File Permissions (7 points)

From Problem 4.5: UNIX treats file directories in the same fashion as files; that is, both are defined by the same type of data structure, called an inode. As with files, directories include a nine-bit protection string. If care is not taken, this can create access control problems.
For example, consider a file with the protection mode 644 (octal) contained in a directory with protection mode 750.

Explain the interpretation of the 9-bit protection string, for both files and directories. (1)
The 9 bit octal string gives a 3 bit value for the file's owner, group, then everyone else.
Each 3-bit sub-field gives the permission to read, write, and execute the file, in that order.

Explain the meaning of the octal protection string of 644 for a file, and 750 for a directory. (2)

- 644 means that the owner can both read and write the file, and everyone else can only read.
- 750 means that the owner can read, write and execute upon the directory. Execute permissions for a directory mean one can `cd` into the directory and `ls -l` on its contents. The members of the group will be able to read and execute, but not write (i.e. create/delete files)

Do an experiment: Make a directory with permissions 773. Can non-owner non-group users list the contents of the directory? Can such users create new files in the directory? (1)
773 gives the outsider the ability to create and read new files, but they cannot ls.

Consider the ls output below.
```
bash  -rw-r--r--  1 root     root 1.6M Sep 25 16:58 gosh.tar.gz
```
Explain in detail what each element this line means. (1)
read/write perms for owner, read for everyone else
1 hard link to the underlying inode
`root` is the owner of the file
`root` is the group of the file
it is 1.6 MB large
last edited on September 25
it's called gosh.tar.gz, which means its a tarred and gzipped (compressed) file.

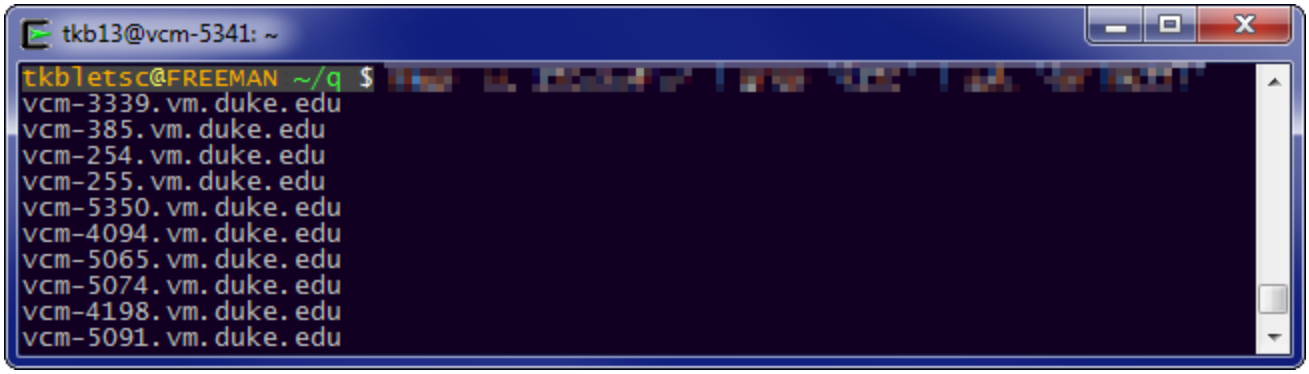
What command would you use to change the access rights to allow any user to edit the file? (1)
```
sudo chmod a+w gosh.tar.gz
```

What command would you use to take ownership of this file from root to yourself? (1)
```
sudo chown $(whoami)
```

Question 3: Data Processing (16 points)

Task 1: Filter nmap output (4 points)

Using any combination of the tools described in class or listed above, on your Linux VM, write a single command with pipes that will parse out only the registered hostnames from an Nmap List scan of the 152.3.64. **subnet.***
Your output should not include those hosts without a hostname or any other extraneous output. Give the full command and a small sample (5 hosts) of output. Do not use `-sT` or `-sS` in the Nmap Scan. A sample screenshot (with command blurred out of course) is shown below:



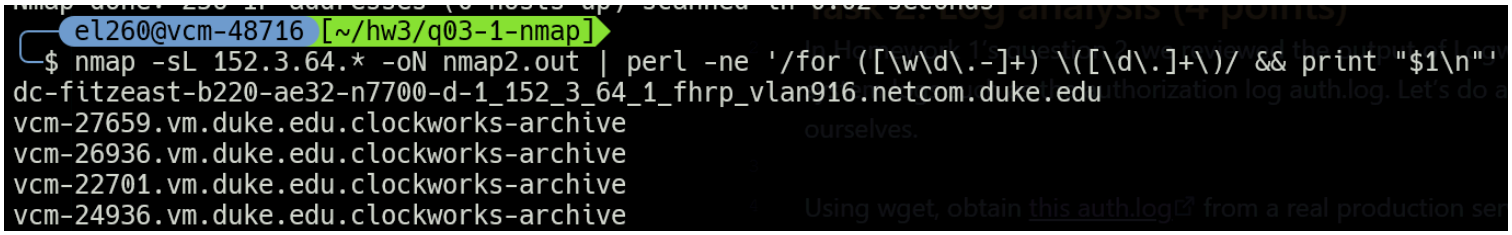
An attacker might use output like the above to better understand a target environment or to feed to another attack tool, especially after they've gained a foothold to an internal network.

a snippet of the output of a raw `nmap -sL 152.3.64.*` gives:

```
Nmap scan report for 152.3.64.212
Nmap scan report for vcm-20664.vm.duke.edu.clockworks-archive (152.3.64.213)
Nmap scan report for 152.3.64.214
Nmap scan report for 152.3.64.215
Nmap scan report for 152.3.64.216
```

So now I want to write a regex that filters for the hostname and ignores ip addresses.
A quick way to do this (in my eyes) is to just filter everything until the parentheses, because that doesn't exist in the other lines without registered hostnames.

```
$ nmap -sL 152.3.64.* -oN nmap2.out | perl -ne '/for ([\w\d\.-]+) \([\d\.]+\)/ && print
"$1\n"'
```



Task 2: Log analysis (4 points)

In Homework 1's question 2, we reviewed the output of Logwatch. That tool analyzes system logs such as the authorization log auth.log. Let's do a bit of similar analysis ourselves.

Using wget, obtain this auth.log from a real production server on the internet.

Write a single command that identifies all the unique IP addresses that tried and failed to login using the invalid user 'admin'. Post a screenshot.

getting file:

```
$ wget http://people.duke.edu/~tkb13/courses/ece560-2025fa/homework/hw3/auth.log
```

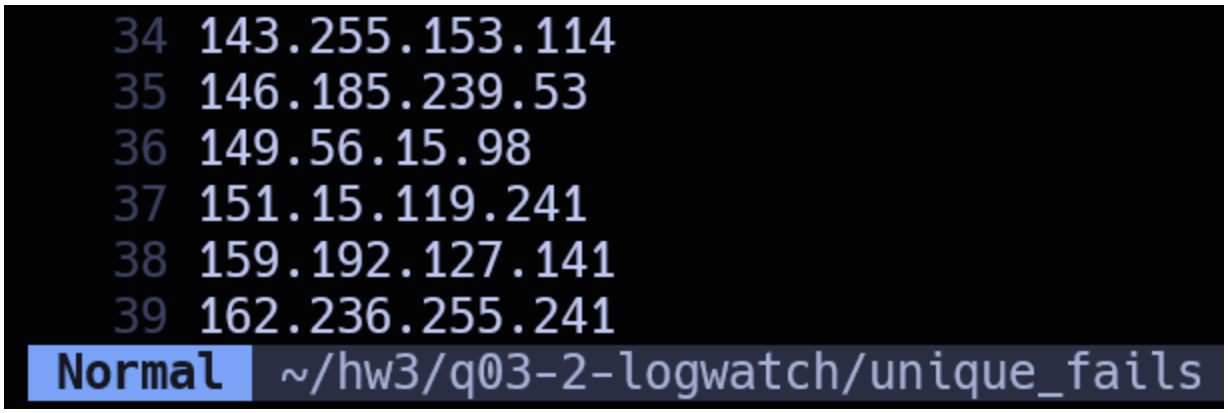
found example of failed login:

```
  346   Aug 12 08:41:47 dlsprod sshd[18168]: Invalid user admin from 186.47.169.18
      1 Aug 12 08:41:47 dlsprod sshd[18168]: input_userauth_request: invalid user admin
[preauth]                                                                    2 Aug 12
08:41:47 dlsprod sshd[18168]: pam_unix(sshd:auth): check pass; user unknown
      3 Aug 12 08:41:47 dlsprod sshd[18168]: pam_unix(sshd:auth): authentication
failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=186.47.169.18
      4 Aug 12 08:41:49 dlsprod sshd[18168]: Failed password for invalid user admin
from 186.47.169.18 port 55681 ssh2
      5 Aug 12 08:41:49 dlsprod sshd[18168]: Connection closed by 186.47.169.18 port
55681 [preauth]
```

looks like since we only care about failed admin logins, we can `awk` out the lines that say "Invalid user admin from ...."
and then filter for uniqueness.
oops I decided to use perl because it has capture group support!

```
$ perl -ne '/Invalid user admin from ([\d\.]+)/ && print "$1\n"' auth.log | sort -u >>
unique_fails
```

wrote it all to a file because it was kind of a big output:



Task 3: Web app needle in a haystack (8 points)

A mock web application has been set up here:
http://target.colab.duke.edu/app/
The term "web application" here is a misnomer: for safety reasons, there's no actual server-side code.

Rather, there's "lorem ipsum" content with hyperlinks that branch out to much of the same, with a few hidden things mixed in.

Using shell-based tools, identify the two pages that have an HTML `<form>` tag and give the sequence of clicks needed to find these pages from the start. Show your work.

TLDR:

Form 1: https://target.colab.duke.edu/app/X-2ef7d6ec1e6416c5ac1c7ab45d2b023a.html
quisquam->est->Neque->adipisci
Form 2: https://target.colab.duke.edu/app/X-2ef7d6ec1e6416c5ac1c7ab45d2b023a.html
non -> non -> quisquam -> labore
Fat Dog:
https://target.colab.duke.edu/app/X-d9ec51ce46005ac4fea99127e1ece66e.html
quisquam->ut->non->numquam
Unicorn: https://target.colab.duke.edu/app/X-691fe1e64eb42b73397c898b2d546b79.html
quisquam->ipsum->quisquam (ut sed.)->**dolor**

the actual process

This can be a recursive search I guess?
wait! wget does recursive downloading!
So this downloads up to 5 links deep of the html pages.
we can then grep for html form tags:

```
$ wget --recursive --accept html,jpg,jpeg,png,webp https://target.colab.duke.edu/app/

$ grep -r form
target.colab.duke.edu/app/X-42408112a81b446d2760c42da749ad30.html:<form><input
type=text value='Sensitive vulnerable stu
ff'><input type=submit></form>
target.colab.duke.edu/app/X-2ef7d6ec1e6416c5ac1c7ab45d2b023a.html:<form><input
type=text value='Sensitive vulnerable stu
ff'><input type=submit></form>
target.colab.duke.edu/app/index.html:For ECE560, Computer and Information Security,
Prof. Tyler Bletsch.
```

Problem: it downloaded everything into a single directory. I suppose I can trace these links backwards though, just have a script that continues grepping for the current url in the directory until it ends up at `index.html`
grep for line containing the file name, get the file name and put the whole line into a new file.

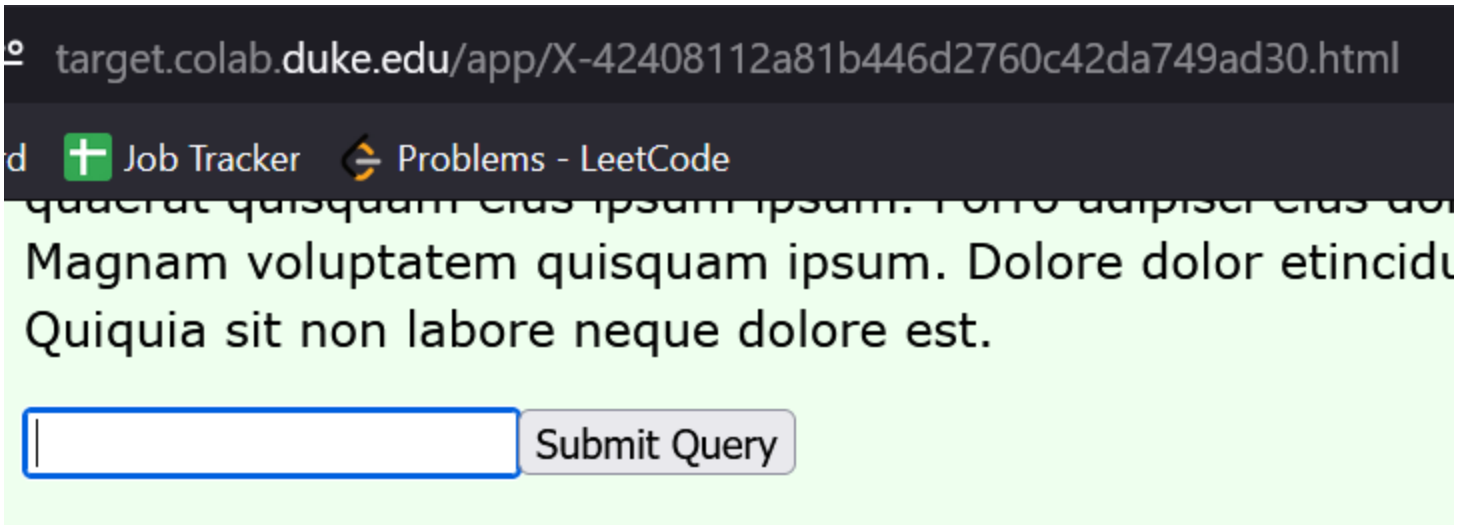
doing this manually first:
(because wget has a default maximum depth of 5, it shouldn't be terrible to just copy/paste things manually.)

```
el260@vcm-48716 [~/.../wget/target.colab.duke.edu/app]
$ grep -r form
X-42408112a81b446d2760c42da749ad30.html:<form><input type=text value='Sensitive vulnerable stuff'><input type=submit></form>
X-2ef7d6ec1e6416c5ac1c7ab45d2b023a.html:<form><input type=text value='Sensitive vulnerable stuff'><input type=submit></form>
index.html:For ECE560, Computer and Information Security, Prof. Tyler Bletsch.
el260@vcm-48716 [~/.../wget/target.colab.duke.edu/app]
$ grep -r X-42408112a81b446d2760c42da749ad30.html
X-55fa3419d5a205551f17e5bf59da6424.html:<p>Eius etincidunt eius sed eius. Porro numquam eius sed dolor neque aliquam. Porro neque neque dolorem quisquam t
pora. Sed est numquam aliquam non. Quisquam etincidunt non ut ut consectetur consectetur. Labore labore <a href='X-42408112a81b446d2760c42da749ad30.html'>
bore</a> eius adipisci dolorem. Adipisci non consectetur sit numquam sit. Numquam sit neque sed.</p>
el260@vcm-48716 [~/.../wget/target.colab.duke.edu/app]
$ grep -r X-55fa3419d5a205551f17e5bf59da6424.html
X-d1bcf79d6b085c032579eae049ede398.html:<p>Quisquam non est dolor dolorem porro. Sed eius voluptatem est porro. Labore amet dolorem dolorem sit. Dolor qui
uam quisquam est magnam. Etincidunt sit dolor ut consectetur porro neque numquam. Voluptatem modi sit quaerat numquam dolorem modi. Magnam velit eius volu
atem eius sed quiquia. Sed sed dolor adipisci <a href='X-55fa3419d5a205551f17e5bf59da6424.html'>quisquam</a>. Tempora porro neque sit.</p>
el260@vcm-48716 [~/.../wget/target.colab.duke.edu/app]
$ grep -r X-d1bcf79d6b085c032579eae049ede398.html
X-e7343ee66f68528c1b9c947a0f3c169f.html:<p>Consectetur ipsum modi dolor. Aliquam neque adipisci quisquam modi modi eius etincidunt. Est velit ipsum sit qu
uia consectetur non. Quisquam dolor porro voluptatem neque. Dolor consectetur ut voluptatem. Ut labore magnam magnam etincidunt adipisci. Dolorem voluptat
 ipsum eius est. Ut velit <a href='X-d1bcf79d6b085c032579eae049ede398.html'>non</a> numquam eius tempora dolore dolore.</p>
el260@vcm-48716 [~/.../wget/target.colab.duke.edu/app]
$ grep -r X-e7343ee66f68528c1b9c947a0f3c169f.html:<p>Consectetur ipsum modi dolor. Aliquam neque adipisci quisquam modi modi eius etincidunt. Est velit
sum sit quiquia sed sit. Etincidunt consectetur non etincidunt dolorem dolore quiquia sed sit. Etincidunt consectetur non etincidunt dolorem dolore <a href='X-e7343ee66f68528c1b9c947a0f3c169f.html'>non</a>. Quisquam quisquam sed amet aliquam voluptatem est consectetur. Labore labore eius ut.</p>
el260@vcm-48716 [~/.../wget/target.colab.duke.edu/app]
$
```

So it seems we click on:

non -> **non** -> **quisquam** -> **labore** -> page with form.

Which indeed gets us to our target page:



I then wrote a script file to do this automatically by grepping for the concerned files:

```
$ grep -r form
X-42408112a81b446d2760c42da749ad30.html:<form><input type=text value='Sensitive
vulnerable stuff'><input type=submit></form>
X-2ef7d6ec1e6416c5ac1c7ab45d2b023a.html:<form><input type=text value='Sensitive
vulnerable stuff'><input type=submit></form>
$ ../../../backtracking.sh X-42408112a81b446d2760c42da749ad30.html
$ ../../../backtracking.sh X-2ef7d6ec1e6416c5ac1c7ab45d2b023a.html
```

```
# backtracking.sh
page=$1

while [[ $page != "index.html" ]] do
    line=$(grep -r $page --exclude="*.trace")
    echo $line | tee -a $1.rev.trace
    page=$(echo $line | perl -ne '/^(.*?):/ && print $1')
done
```

```
# reverse order of lines to make sequence of clicks make sense
tac $1.rev.trace > $1.trace
rm $1.rev.trace
```

and now for my final trick, I also did this for `fd.jpg`, which I found had been downloaded from the server as well. The `tail` at the end was a trick I saw described in a stackoverflow post that I x-ed out of before saving the url, but it also shows the title of each file.

```
$ ls | grep trace | xargs tail -v -n +1
==> X-2ef7d6ec1e6416c5ac1c7ab45d2b023a.html.trace <==
index.html:<p>Dolor neque labore porro aliquam est numquam. Sit voluptatem dolor dolore
velit labore consectetur. Eius etincidunt porro labore sit dolorem. Neque modi non <a
href='X-676a03565c0b7fe037e1f73484c27829.html'>quisquam</a> eius ipsum sit. Dolore est
adipisci voluptatem etincidunt aliquam. Voluptatem modi ipsum numquam est modi labore.
Aliquam quisquam numquam quiquia dolor magnam voluptatem. Voluptatem sed ut quaerat.
Adipisci sit aliquam consectetur consectetur. Etincidunt etincidunt numquam dolorem
porro magnam numquam.</p>
X-676a03565c0b7fe037e1f73484c27829.html:<p>Amet quisquam non porro dolorem tempora.
Dolorem non quisquam aliquam non dolorem quaerat est. Dolor labore dolor quiquia sed
labore etincidunt. Numquam est consectetur modi ipsum velit dolore magnam. Non quisquam
eius magnam. Etincidunt <a href='X-46cd7fad086632a9bb75c0faa602baea.html'>est</a> porro
sed est non consectetur eius. Dolor porro ipsum dolore neque quiquia. Est magnam
consectetur eius neque adipisci numquam.</p>
X-46cd7fad086632a9bb75c0faa602baea.html:<p>Modi adipisci porro quisquam sit non
tempora. Tempora porro magnam modi consectetur velit tempora est. Aliquam voluptatem
tempora numquam porro. Porro est quisquam quaerat labore dolore modi. Magnam ipsum non
est porro est etincidunt porro. <a href='X-
2ddf408d230313a98bea27a7215f1322.html'>Neque</a> adipisci neque aliquam. Sed labore
quiquia aliquam. Dolorem quiquia dolor consectetur magnam. Labore quiquia ipsum sed
amet quiquia dolore etincidunt. Etincidunt etincidunt modi quaerat amet quisquam.</p>
X-2ddf408d230313a98bea27a7215f1322.html:<p>Labore dolor aliquam quiquia ipsum tempora.
Velit ut aliquam labore neque magnam. Magnam sed numquam porro. Porro dolorem <a
href='X-2ef7d6ec1e6416c5ac1c7ab45d2b023a.html'>adipisci</a> labore dolor. Quaerat
voluptatem modi quiquia dolor quaerat. Amet dolor ut ut labore consectetur.</p>

==> X-42408112a81b446d2760c42da749ad30.html.trace <==
index.html:<p>Dolorem tempora voluptatem porro est modi consectetur. Consectetur dolore
quiquia sed sit. Etincidunt consectetur non etincidunt dolorem dolore <a href='X-
e7343ee66f68528c1b9c947a0f3c169f.html'>non</a>. Quisquam quisquam sed amet aliquam
voluptatem est consectetur. Labore labore eius ut.</p>
X-e7343ee66f68528c1b9c947a0f3c169f.html:<p>Consectetur ipsum modi dolor. Aliquam neque
adipisci quisquam modi modi eius etincidunt. Est velit ipsum sit quiquia consectetur
non. Quisquam dolor porro voluptatem neque. Dolor consectetur ut voluptatem. Ut labore
magnam magnam etincidunt adipisci. Dolorem voluptatem ipsum eius est. Ut velit <a
href='X-d1bcf79d6b085c032579eae049ede398.html'>non</a> numquam eius tempora dolore
dolore.</p>
X-d1bcf79d6b085c032579eae049ede398.html:<p>Quisquam non est dolor dolorem porro. Sed
eius voluptatem est porro. Labore amet dolorem dolorem sit. Dolor quisquam quisquam est
```

```
magnam. Etincidunt sit dolor ut consectetur porro neque numquam. Voluptatem modi sit
quaerat numquam dolorem modi. Magnam velit eius voluptatem eius sed quiquia. Sed sed
dolor adipisci <a href='X-55fa3419d5a205551f17e5bf59da6424.html'>quisquam</a>. Tempora
porro neque sit.</p>
X-55fa3419d5a205551f17e5bf59da6424.html:<p>Eius etincidunt eius sed eius. Porro numquam
eius sed dolor neque aliquam. Porro neque neque dolorem quisquam tempora. Sed est
numquam aliquam non. Quisquam etincidunt non ut ut consectetur consectetur. Labore
labore <a href='X-42408112a81b446d2760c42da749ad30.html'>labore</a> eius adipisci
dolorem. Adipisci non consectetur sit numquam sit. Numquam sit neque sed.</p>

==> fd.jpg.trace <==
index.html:<p>Dolor neque labore porro aliquam est numquam. Sit voluptatem dolor dolore
velit labore consectetur. Eius etincidunt porro labore sit dolorem.
Neque modi non <a href='X-676a03565c0b7fe037e1f73484c27829.html'>quisquam</a> eius
ipsum sit. Dolore est adipisci voluptatem etincidunt aliquam. Voluptatem
modi ipsum numquam est modi labore. Aliquam quisquam numquam quiquia dolor magnam
voluptatem. Voluptatem sed ut quaerat. Adipisci sit aliquam consectetur consectetur.
Etincidunt etincidunt numquam dolorem porro magnam numquam.</p>
X-676a03565c0b7fe037e1f73484c27829.html:<p>Eius sed dolorem <a href='X-
995185d82572d3754a37df893d580893.html'>ut</a>. Velit dolorem numquam sit velit. Est
quisquam magnam sed voluptatem porro dolore. Magnam dolor consectetur adipisci. Ipsum
quiquia sed ut. Aliquam voluptatem amet adipisci voluptatem quaerat tempora. Neque
aliquam eius adipisci ut quisquam quaerat neque.</p>
X-995185d82572d3754a37df893d580893.html:<p>Ipsum labore quiquia sed. Labore consectetur
amet adipisci adipisci. Dolorem consectetur magnam tempora. Eius ipsum aliquam est
dolorem. Etincidunt magnam est quisquam dolore numquam numquam quaerat. Dolor labore
dolore neque neque <a href='X-33b6bc70b521fbcb10b5bbdf8461d0f1.html'>non</a> dolore.
Sed labore non magnam. Amet dolore adipisci velit labore. Est adipisci voluptatem sit
aliquam quisquam dolore est. Velit sed ne
que neque dolor.</p>
X-33b6bc70b521fbcb10b5bbdf8461d0f1.html:<p>Velit ipsum numquam sed aliquam velit.
Dolorem est magnam etincidunt amet. Voluptatem dolorem porro ipsum dolore. Porro neque
labore est sed. Porro labore ut modi tempora tempora tempora dolore. Velit numquam
neque amet. Amet amet ut sit aliquam. Dolorem aliquam <a hre
f='X-d9ec51ce46005ac4fea99127e1ece66e.html'>numquam</a> magnam eius modi neque tempora.
Non labore dolorem quisquam adipisci quiquia. Adipisci ipsum velit non ut numquam dolor
neque.</p>
X-d9ec51ce46005ac4fea99127e1ece66e.html:<p><img src='fd.jpg'></p>
```

After `wget`ing again with a depth of 10 and not seeing a difference in file size and no new image files, I suspected that this unicorn was perhaps made in ASCII art rather than pixels, so I did a grep for something that looked vaguely similar to a horn:

```
$ grep -r '/\\'
X-691fe1e64eb42b73397c898b2d546b79.html:                              `__/\_ --
(/|___/-/
X-691fe1e64eb42b73397c898b2d546b79.html:|  | ( (  \   |      ,/\'__/'/         |     |
X-691fe1e64eb42b73397c898b2d546b79.html:| | \  \_. \,                          \___/\
```

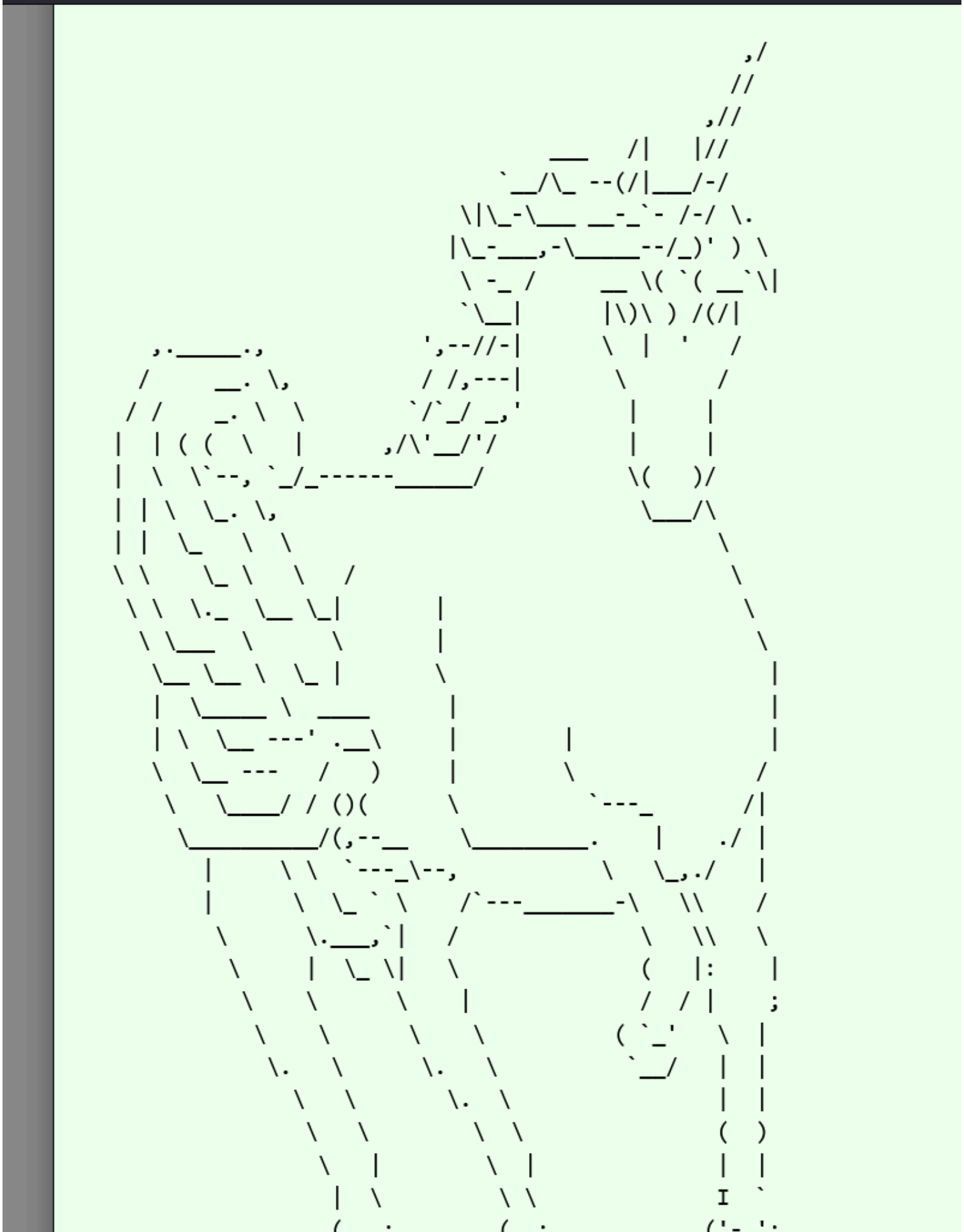
Looks like ASCII art to me.

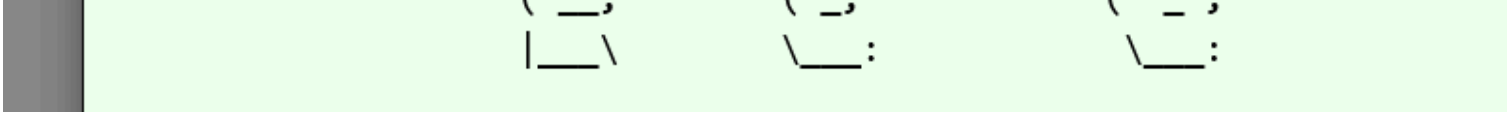
```
$ ../../../backtracking.sh X-691fe1e64eb42b73397c898b2d546b79.html
index.html:<p>Dolor neque labore porro aliquam est numquam. Sit voluptatem dolor dolore
velit labore consectetur. Eius etincidunt porro labore sit dolorem. Neque modi non <a
href='X-676a03565c0b7fe037e1f73484c27829.html'>quisquam</a> eius ipsum sit. Dolore est
adipisci voluptatem etincidunt aliquam. Voluptatem modi ipsum numquam est modi labore.
Aliquam quisquam numquam quiquia dolor magnam voluptatem. Voluptatem sed ut quaerat.
Adipisci sit aliquam consectetur consectetur. Etincidunt etincidunt numquam dolorem
porro magnam numquam.</p>
X-676a03565c0b7fe037e1f73484c27829.html:<p>Est quisquam quisquam quiquia. Quaerat amet
amet voluptatem quiquia <a href='X-df631202204552690702b06aacb54d60.html'>ipsum</a>
labore neque. Adipisci aliquam dolor amet voluptatem. Non numquam dolorem adipisci sed
dolor eius quaerat. Eius eius etincidunt porro. Velit labore magnam modi eius est
magnam. Numquam quisquam amet sit dolorem quisquam eius. Labore numquam tempora
quisquam ipsum. Ipsum voluptatem amet numquam sed dolor numquam.</p>
X-df631202204552690702b06aacb54d60.html:<p>Velit labore consectetur porro voluptatem
etincidunt amet voluptatem. Quiquia neque adipisci etincidunt est <a href='X-
737725720366ddb2ae67197f41a8fdf3.html'>quisquam</a> ut sed. Consectetur quisquam
quisquam numquam amet. Neque magnam eius dolore quaerat sit non ipsum. Velit neque
magnam consectetur quiquia modi consectetur. Est dolor neque non quaerat consectetur
labore. Numquam amet quaerat numquam.</p>
X-737725720366ddb2ae67197f41a8fdf3.html:<p>Quisquam modi <a href='X-
691fe1e64eb42b73397c898b2d546b79.html'>dolor</a> etincidunt quisquam sed. Porro
adipisci dolore modi amet labore. Numquam ipsum dolore non dolorem modi ipsum porro.
Dolorem amet dolore sed eius. Eius dolore eius amet voluptatem. Velit quaerat amet ut
voluptatem est non. Non porro tempora eius aliquam ipsum sit. Sit sit etincidunt sit
aliquam non consectetur voluptatem. Dolore voluptatem velit velit quaerat est aliquam
ut. Voluptatem aliquam ipsum voluptatem.</p>
```

so we go: **quisquam->ipsum->quisquam** (ut sed.)->**dolor**
(which translates to whoever, itself, whoever, pain, which sounds like the type of thing I was muttering to myself while I was debugging my bash)

target.colab.duke.edu/app/X-691fe1e64eb42b73397c898b2d546b79.html

Discord Job Tracker Problems - LeetCode





Majestic.

OPTIONAL: For up to 4 points of extra credit, find:

1. A fat dog

2. A unicorn

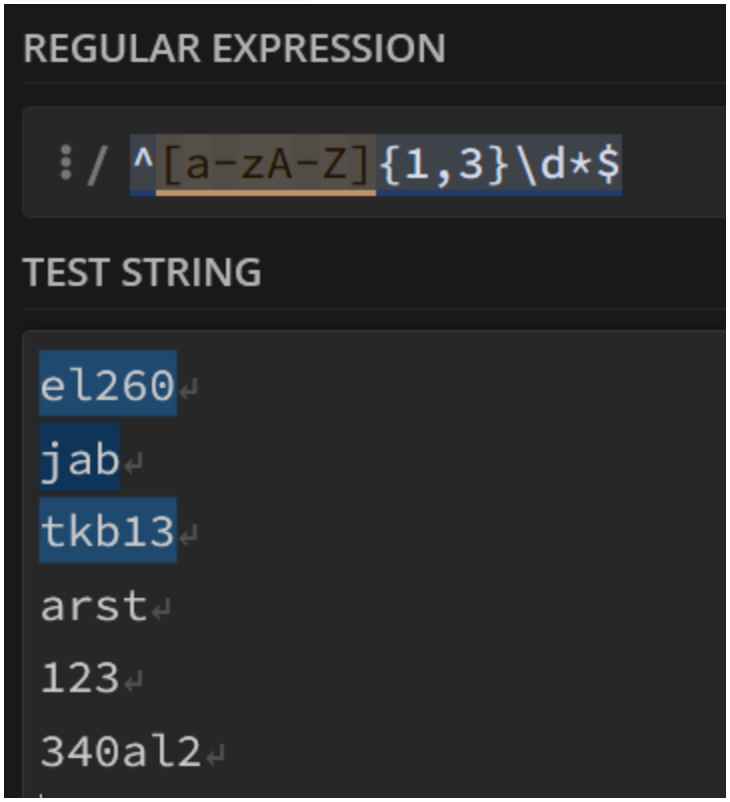
As with the main question, give the page with the content and the click sequence needed to get there from the start. Show your work.

Question 4: Regular expressions (9 points)

NetID validator (3 points)

Develop a regular expression that matches if and only if the input is a valid NetID (1-3 letters, 1 or more digits).

```
^[a-zA-Z]{1,3}\d*$
```



URL parser (3 points)

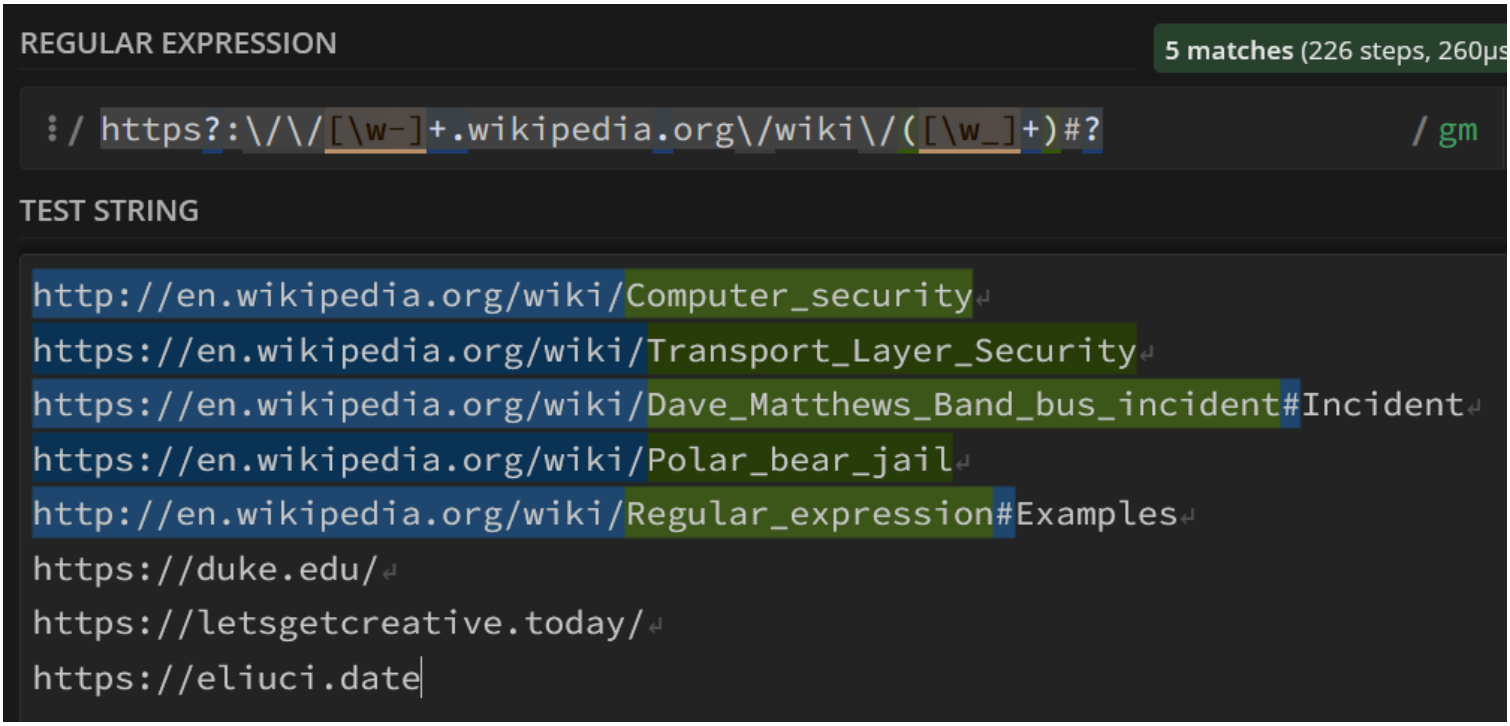
Develop a regular expression that, for a given URL, matches if and only if it is a Wikipedia page, and it captures just the article part of the URL. It should work for HTTP and HTTPS.

Examples:

URL	Match?	Match group 1
http://en.wikipedia.org/wiki/Computer_security	yes	Computer_security
https://en.wikipedia.org/wiki/Transport_Layer_Security	yes	Transport_Layer_Security
https://duke.edu/	no	-

```
https?:\/\/[\w-]+.wikipedia.org\/wiki\/([\w_]+)#?
```

and proof of it working:



Pi Digits (3 points)

Develop a shell command to print the MD5 hash of pi (π) up to and including the first appearance of decimal digits “12345”. To help check your work, the last byte of the answer is 0x02. You can download the value of pi, no need to compute it. For actually employing the regex, you can use grep with `-P` and other options, perl, Python, or other tools. Note: newlines or other whitespace (such as those appended by most terminal programs by default) should NOT be included in your hash. **Show the command you used and the full resulting hash.**

Ok. So probabilistically, '12345' should show up in pi once every 100k digits. I will start with the 100k digit file.
Running `$ grep 12345 100000.txt -c` (which counts the number of occurrences) tells us that there is only 1 occurrence of pi within the first 100k digits of pi, so we're safe here.
Surely, we should be able to just capture only up to 12345 and pipe that into a md5 function.

```
$ grep -oP "(.*12345)" 100000.txt | tr -d '\n' | md5sum
c071744ca7b120fba43d71a219ae1402  -
```

explanation:
`-o` outputs only the portion of the input line that matches the pattern.
`-P` turns the patterns into perl-compatible ones. This makes the process not hang.
`tr` is a command that translates/deletes characters. Here, I've directed it to delete the trailing newline.

Question 5: Manual user management (6 points)

In order to better understand the raw underpinnings of user authentication, we're going to manipulate the Linux authentication system manually without the aid of the usual tools. One reason an attacker might do this is to reduce the probability of detection, as pre-fabricated tools like adduser might trigger log events and review by admins.

WARNING: we're going to be manually editing low level authentication files, and if you make a mistake, it can render your Linux VM inaccessible. Ensure you have no files you need on there, as you'll have to blow away and re-create the VM in this case.

First, research the format of the /etc/passwd and /etc/shadow files.

What's the purpose of /etc/passwd? What fields are stored in it?
According to ArchWiki:
`/etc/passwd` is a plaintext file used to store user information.
It stores data in a colon-separated format as such:
`account:password:UID:GID:GECOS:directory:shell`
where:
`account` : the user name
`password` : the user's password. Typically, this should be `x`, which indicates the corresponding password is stored in the shadow file.
`UID` : the numerical id of the user, usually > 1000
`GID` : the primary group id of the user.
`GECOS` : optional field that usually contains the user's full name.
`directory` : the directory that gets mapped to that user's `$HOME` variable (`~`)
`shell` : the path to the user's default command shell, such as bash.

What's the purpose of /etc/shadow? What fields are stored in it?
According to man7.org:
`/etc/shadow` is a file containing the password information for the system's accounts.
This file is meant to be unreadable to user accounts.
It consists of colon-separated rows of user information:
`login name` : account name that exists on the system
`encrypted password` : usually encrypted with `crypt(3)`. This spits out an encrypted password string delimited by dollar signs, which denotes the algorithm used, the salt, and the hashed string.
`date of last password change` : the date the password was last changed. If left empty, indicates password expiration is disabled on this system.
`minimum password age` : the number of days the user must wait before changing their password again
`maximum password age` : if a password is older than this number of days, the user will be prompted to change it after their next login.
`password warning age` : if a user's password is this number of days close to expiring, they will be warned to change their password.
`password inactivity period` : the number of days after password expiration where the password is still

able to be used for login. Afterwards, the user must contact a sysadmin to log in.
`account expiration date` : the date after which a user's account will be unable to log in.
`reserved for future use` : title.

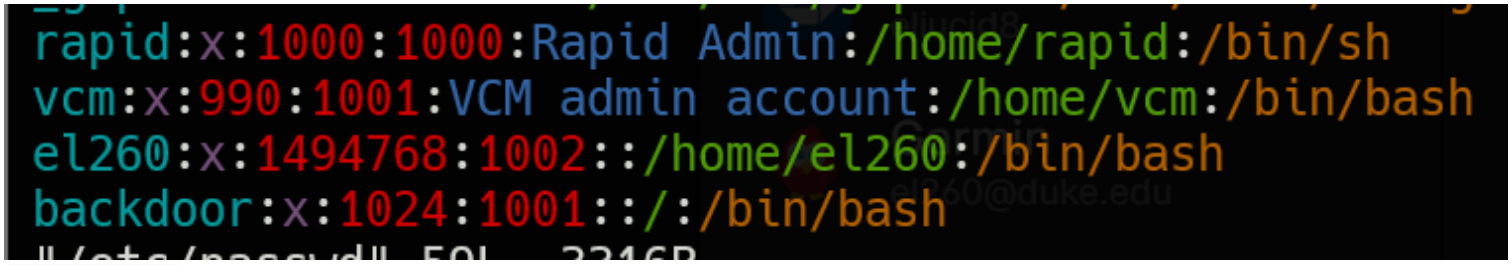
Second, let's add a user by hand. Use sudo to become root (e.g. `sudo -i`). For all steps that say "as root", you'll use this session. Note: it's wise to keep a spare terminal window logged in as root in the background, so if you screw something up with logins, you can fix it without trashing the VM.

In order to add a user, we'll need a password hash suitable for use on Linux. On your Kali VM, the tool `mkpasswd` is pre-installed. Run it, and type in a reasonably strong unique password you can remember. The output will be a suitable salted hash.

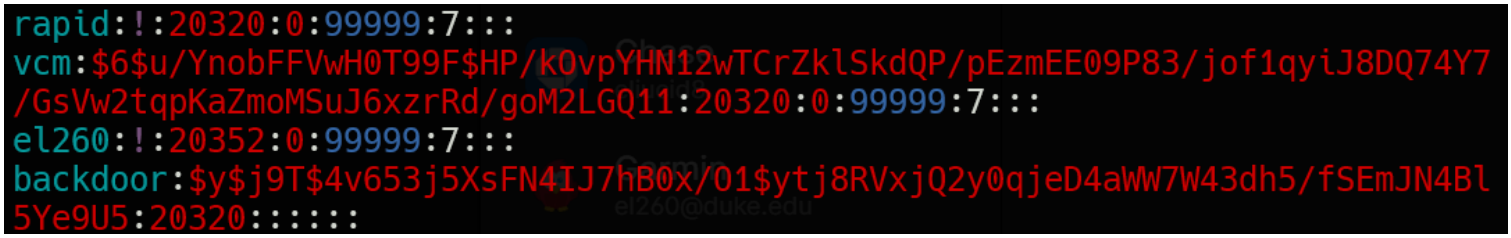
hash: `$y$j9T$4v653j5XsFN4IJ7hB0x/O1$ytj8RVxjQ2y0qjeD4aWW7W43dh5/fSEmJN4Bl5Ye9U5`

As root on your Linux VM, edit `/etc/passwd` and `/etc/shadow` to add a new user called backdoor. For UID, pick any unused number above 1000, and pick reasonable values for the other fields (experimenting as needed). For the password hash, use the one generated above. You should now be able to SSH into your VM as the backdoor user with the chosen password. ***Show your changes to `/etc/passwd` and `/etc/shadow` as well as the successful login.***

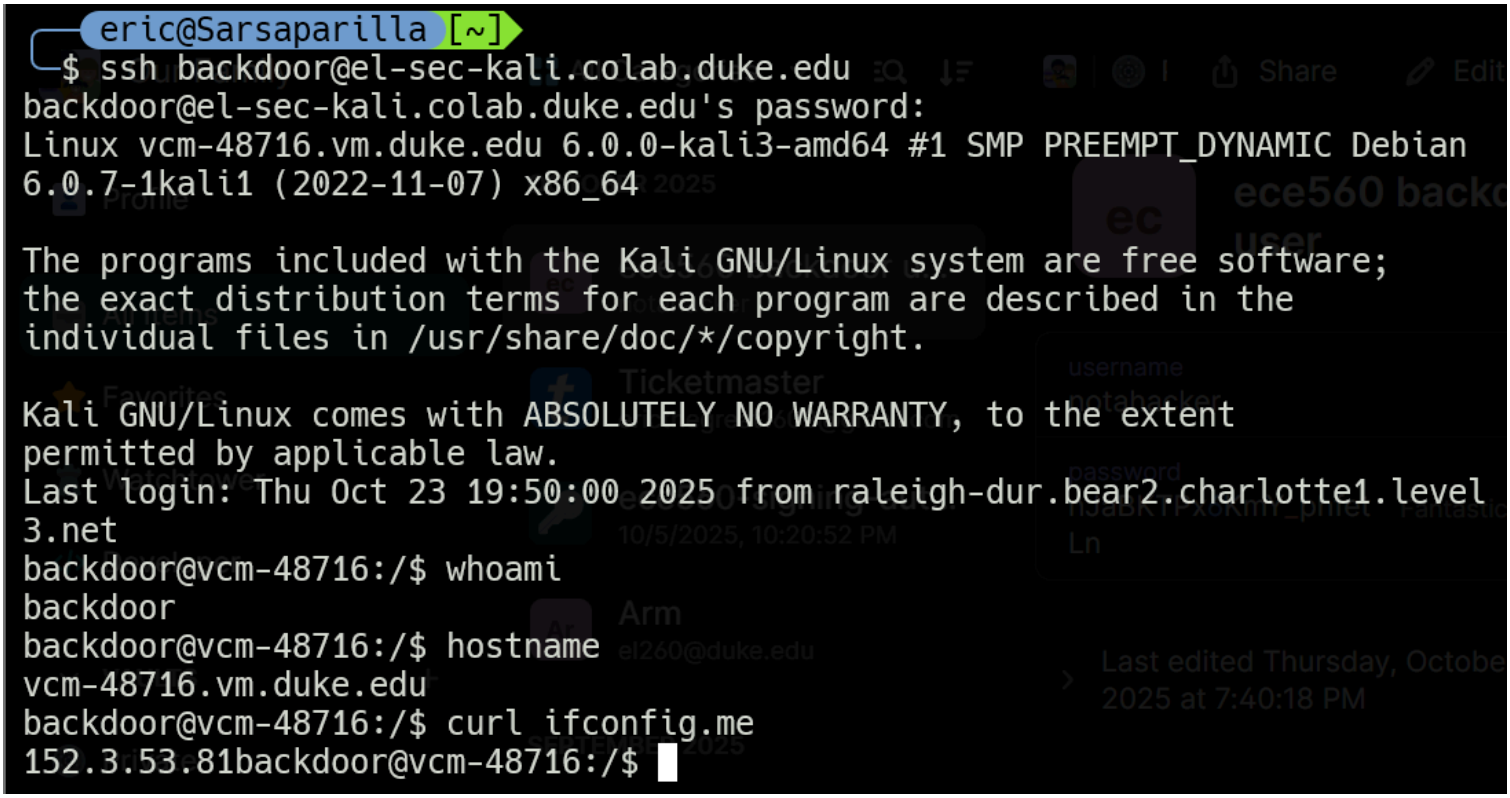
`etc/passwd` :



`etc/shadow` :



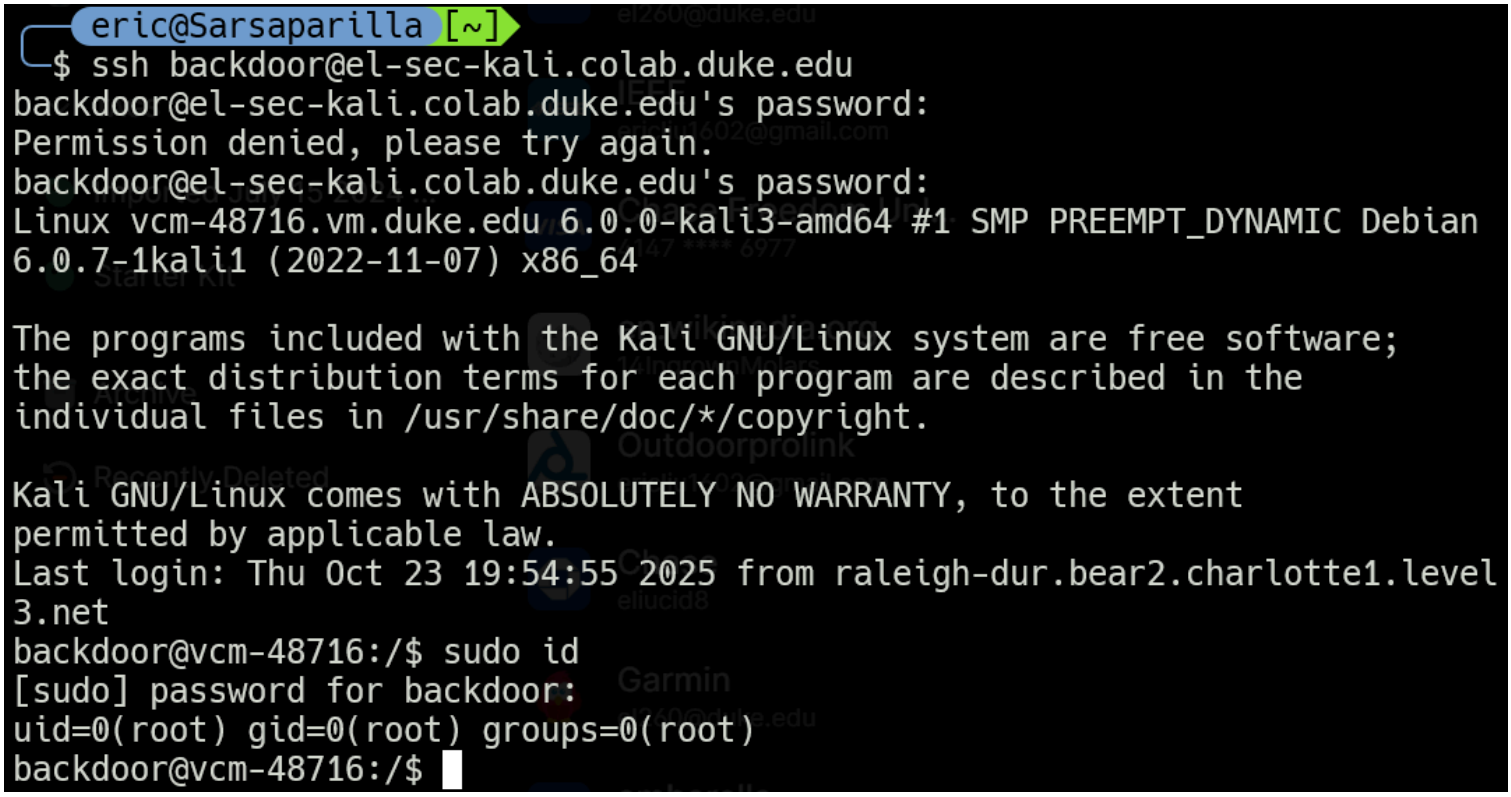
backdoor login:



You've now added a new user account. It doesn't have a real home directory, but that doesn't matter to an attacker who's more interested in sustained access than storing documents. Currently, however, this user is unprivileged, unable to use sudo or editing much of anything.

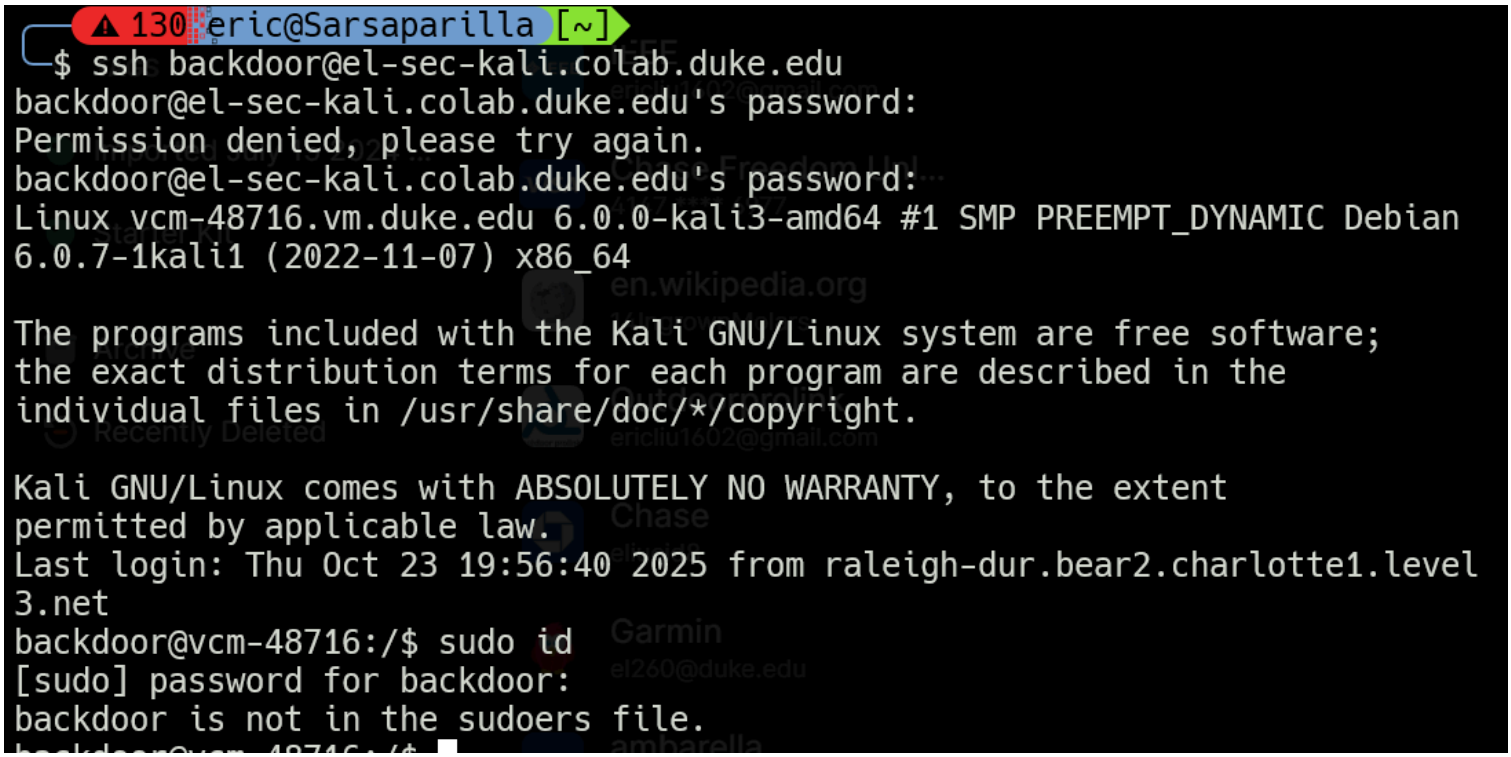
We're going to take two separate approaches to adding privileges to the backdoor user. As root on your Linux VM, let's add the user to the sudo group, which gives it sudo permissions (so the backdoor can become root). This is quite easy -- ***using your original root termal, edit*** `/etc/group` ***, and add backdoor at the end of the "sudo:" line. Open a new SSH connection as backdoor, and verify you can use sudo with*** `sudo id` ***.***

pwned:



There's a more subtle way to gain access, however. To test this, ***first undo the above by removing backdoor from the sudo group, and verify that backdoor no longer has sudo rights.***

unpwned:



Normally, there's a 1:1 mapping between usernames and UID numbers, but we're going to intentionally create a UID collision. ***As root on your Linux VM, edit*** `/etc/passwd`***, and change the UID of the backdoor user to the same number as your NetID (also shown in the passwd file). Ensure that the line for the backdoor user appears after your NetID in the passwd file. Then, in a new terminal, SSH into the Linux VM as backdoor again. If you've done it right, you should find yourself suddenly logged in under your NetID account! Show the edited passwd file (with both your NetID***

and backdoor entries), as well as a successful SSH connection where you logged in as backdoor but became your NetID. Verify that you can use sudo just as your normal NetID login can.

UID conflict:

```
rapid:x:1000:1000:Rapid Admin:/home/rapid:/bin/sh
vcm:x:990:1001:VCM admin account:/home/vcm:/bin/bash
el260:x:1494768:1002::/home/el260:/bin/bash
backdoor:x:1494768:1001::/:/bin/bash
```

here, I ssh as backdoor but log in as el260!

```
eric@Sarsaparilla [~]
$ ssh backdoor@el-sec-kali.colab.duke.edu
backdoor@el-sec-kali.colab.duke.edu's password:
Linux vcm-48716.vm.duke.edu 6.0.0-kali3-amd64 #1 SMP PREEMPT_DYNAMIC Debian
6.0.7-1kali1 (2022-11-07) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Oct 24 00:13:41 2025 from smartchoice68703.s.subnet.rcn.com
el260@vcm-48716:/$ sudo id
[sudo] password for el260:
uid=0(root) gid=0(root) groups=0(root)
el260@vcm-48716:/$
```

Now we've set up an alternate means of logging into the VM as you, but without your password, and we did it without triggering any log file entries about the change. Let's see what log entries are created with regard to login. As root on your Linux VM, observe the content of `/var/log/syslog` ***and*** `/var/log/auth.log` ***during an SSH login as backdoor. If an administrator only looked at syslog, could they tell that a backdoor user was involved in the login?***

here, using `sudo journalctl -u ssh` shows that backdoor is indeed the account logging in.

```
pam_krb5(sshd:auth): authentication failure; logname=backdoor uid=0 euid=0 tty=ssh ruser= rhost>
Accepted password for backdoor from 207.38.206.242 port 15935 ssh2
pam_unix(sshd:session): session opened for user backdoor(uid=1494768) by el260(uid=0)
```

For safety, undo all changes to `/etc/passwd` ***and*** `/etc/shadow` ***when done.***

Question 6: GNU Screen (2 points)

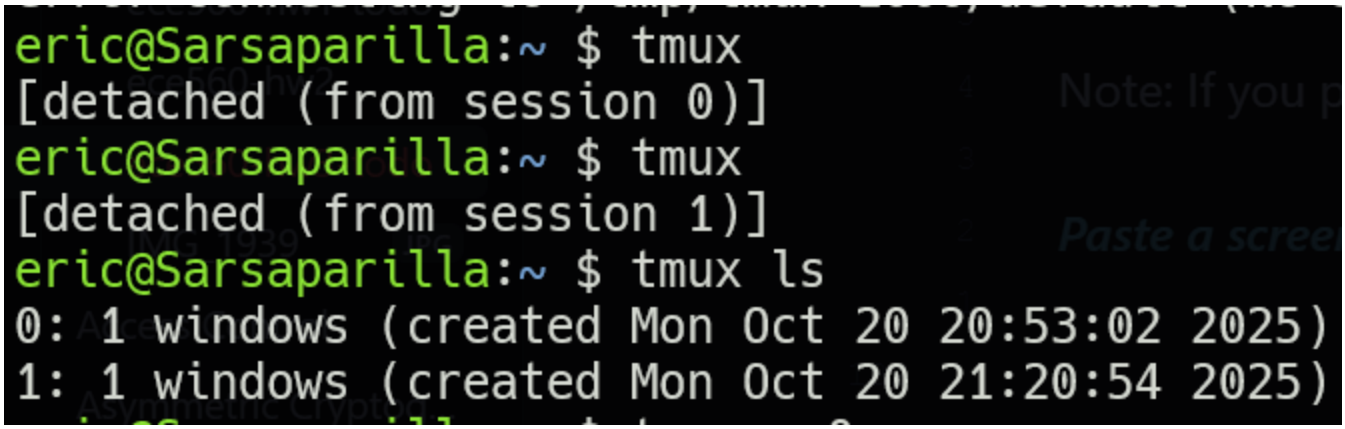
Screen is a full-screen window manager that multiplexes a physical terminal (or GUI terminal window) between several processes, typically interactive shells. When screen is called, it creates a single terminal with a shell in it (or the specified command) and then gets out of your way so that you can use the program as you normally would. Then, at any time, you can create new (full-screen) terminals with other programs in them (including more shells), kill the current window, view a list of the active windows, turn output logging on and off, view the scrollback history, switch between terminals, etc. All terminals run their programs completely independent of each other. Especially useful for us is the fact you can detach a running screen session from your console and let all the associated terminals keep running in the background, then later reattach it to another console. Programs continue to run even when screen is detached.

Review this very brief video intro to screen. Here is a screen quick reference.

This might be useful for the MD5 cracking question below, and it's essential for the problem "John the Ripper" after that. Start a new screen session on your Kali VM. Prove to yourself that you can detach, reattach, and are generally comfortable with screen.

Note: If you prefer another terminal virtualizer, such as tmux, you may use that instead.

Paste a screenshot below of screen --list two separate screen sessions running.



tmux my goat!

Question 7: Cracking MD5 hashes with hashcat (5 points)

There is an old FTP server called Serv-U, a commercial program originally written by CATsoft. In the past, hackers would sometimes install pirated copies on computers they compromise. This allows an attacker to access the file system and run commands on the compromised computer. The FTP server usernames and passwords (used by the attackers) are stored in a configuration file. The passwords are hashed with MD5. Here is a sample file from a compromised machine:

```
[USER=admin|1]
Password=ly5cf2ff593419bcf3d22c62e65a82128a
HomeDir=c:\
AlwaysAllowLogin=1
TimeOut=600
Maintenance=System
Access1=C:\|RWAMELCDP
```

The MD5 hash of the password is on the line that starts with ‘Password=’. The two letters after the equal sign are the salt as ASCII text (shown in orange above) followed by the MD5 hash in hex (shown in blue above). This salt is prepended, i.e. the Serv-U password hash function is md5(salt+pass).

Your Kali VM comes with a hash cracker called hashcat. Read up on how to use it, and have it crack the salted hash above.
Tips:

- Hashcat is optimized for GPUs, but your VM doesn’t have one. Override the resulting error message using the --force flag.
- You will need to create a “hash file” as input; the format of this file is “<hash>:<salt>”.
- The default brute force mode will be fine, and should take around 30 seconds on the Kali VM.
- Found passwords are echoed in <hash>:<salt>:<pass> format and also saved to `~/.hashcat/hashcat.potfile`.

Paste (1) the command executed, (2) a screenshot of the output, and (3) the password itself below.

hash type: 20 = md5(pass.salt)

```
$ hashcat password -m 20 -a 3 -o unhashed --force
```

```
Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 20 (md5($salt.$pass))
Hash.Target......: 5cf2ff593419bcf3d22c62e65a82128a:ly
Time.Started.....: Mon Oct 27 17:20:18 2025, (20 secs)
Time.Estimated...: Mon Oct 27 17:20:38 2025, (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.......: ?1?2?2?2?2?2 [6]
Guess.Charset....: -1 ?l?d?u, -2 ?l?d, -3 ?l?d*!$@_, -4 Undefined
Guess.Queue......: 6/15 (40.00%)
Speed.#1.........: 15854.4 kH/s (10.61ms) @ Accel:512 Loops:128 Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 320241664/3748902912 (8.54%)
Rejected.........: 0/320241664 (0.00%)
Restore.Point....: 143360/1679616 (8.54%)
Restore.Sub.#1...: Salt:0 Amplifier:0-128 Iteration:0-128
Candidate.Engine.: Device Generator
Candidates.#1....: satwnd -> coltho
Hardware.Mon.#1..: Util: 24%

Started: Mon Oct 27 17:19:34 2025
Stopped: Mon Oct 27 17:20:40 2025
```

```
$ cat unhashed
5cf2ff593419bcf3d22c62e65a82128a:ly:manito
```

"manito" is the password.

OPTIONAL: For up to 6 points of extra credit, crack each of the following Serv-U hashes.

You may need to use more time, wordlists, and possibly even a combination wordlist/brute-force attack.

1. `qje39945197dbc380a5ac611fbbb7b5354`
2. `csa2eb704ec429728c1fe15814c006a4cc`
3. `qj847d6d5952baef48fa101d84a18b8de8`

found using: `$ hashcat extra -m 20 -a 0 -o cracked-extra -O --force /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best64.rule`

1. e39945197dbc380a5ac611fbbb7b5354:qj:baseball

found using: `$ hashcat extra -m 20 -a 0 -o cracked-extra -O --force /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/rockyou-30000.rule`

2. 847d6d5952baef48fa101d84a18b8de8:qj:dreamweaver813

Question 8: John the Ripper (6 points)

Note: before doing this question, you should understand the /etc/passwd and /etc/shadow files, as introduced in Question 5.

For this exercise, we will use John the Ripper, which is already installed on your Kali VM. I'll be supplying the shadow file for the exercise.

We will be using the shadow file here: http://people.duke.edu/~tkb13/courses/ece560-2025fa/homework/hw3/shadow

Download this shadow file to your Kali VM. Unlike hashcat, john has pretty smart defaults and wordlists, so you can just run it against the shadow file. To improve performance, you can specify `--fork=\<N>` to run N instances in parallel; set N to the number of CPU cores on your system (you can check with `top`, `cat /proc/cpuinfo`, etc.).

This baby is 4-core enabled!

Run john on the provided shadow file in a screen session (so you can detach, disconnect, then reattach later). Let it run for at least 24 hours. It is not expected to be able to crack all the passwords, but you should get at least 6.

Paste the output of your results with the following command "john -show shadow".
(6 points)

It should look something like the example below:

```
$ john -show shadow
abcuser:apple1:15358:0:99999:7:::
defuser:orange:15364:0:99999:7:::
ghiuser:gpgtest:15373:0:99999:7:::
3 password hashes cracked, 25 left
```

Note: Here, "abcuser" is the username and "apple1" is the password.

ok, here goes nothing:
`$ john shadow --fork=4`
Off to the races!

```
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 282507 el260     39  19   82044  44004   2228 R  97.3   0.5   0:38.32 john-avx-non-om
 282505 el260     39  19   82044  49788   8012 R  97.0   0.6   0:38.51 john-avx-non-om
 282506 el260     39  19   82044  44004   2228 R  95.7   0.5   0:38.24 john-avx-non-om
 282508 el260     39  19   82044  44004   2228 R  95.3   0.5   0:38.30 john-avx-non-om
   2363 el260     20   0   10208   5412   2772 S   1.3   0.1   0:54.26 tmux: server
```

ran it for just over 2 days.

```
4 1g 2:01:50:44  3/3 0.000005g/s 218.6p/s 877.8c/s 877.8C/s lexygigg..lexygy71
1 3g 2:01:50:44  3/3 0.000016g/s 218.6p/s 877.9c/s 877.9C/s thyatjk..thyags7
2 2g 2:01:50:43  3/3 0.000011g/s 218.5p/s 877.4c/s 877.4C/s 14167950..14167139
3 2g 2:01:50:43  3/3 0.000011g/s 218.5p/s 877.5c/s 877.5C/s mahalers3..mahanitce

$ john --show shadow
root:toor:10063:0:99999:7:::
idiot:idiot:10063:0:99999:7:::
al:1einstein:10063:0:99999:7:::
cindy:crawford:10063:0:99999:7:::
dieter:curiake:10063:0:99999:7:::
dean:astalis:10063:0:99999:7:::
sgtpepper:ringo:10063:0:99999:7:::

7 password hashes cracked, 4 left
```

Question 9: Evaluating MFA approaches (8 points)

Review this article on a data theft attack against content aggregator site Reddit and answer the following.

1. **What form of Multi-Factor Authentication (MFA) was deployed at Reddit?**
2. ***What are two ways in which the attacker may have gained access to the second factor?***
3. ***What are two alternative forms of MFA that could be deployed instead?***
4. ***Identify another attack in the last 5 years in which SMS-based MFA was compromised. Describe what happened, and provide one or more citations of where you found this info.***

Question 10: SSH with MFA (5 points)

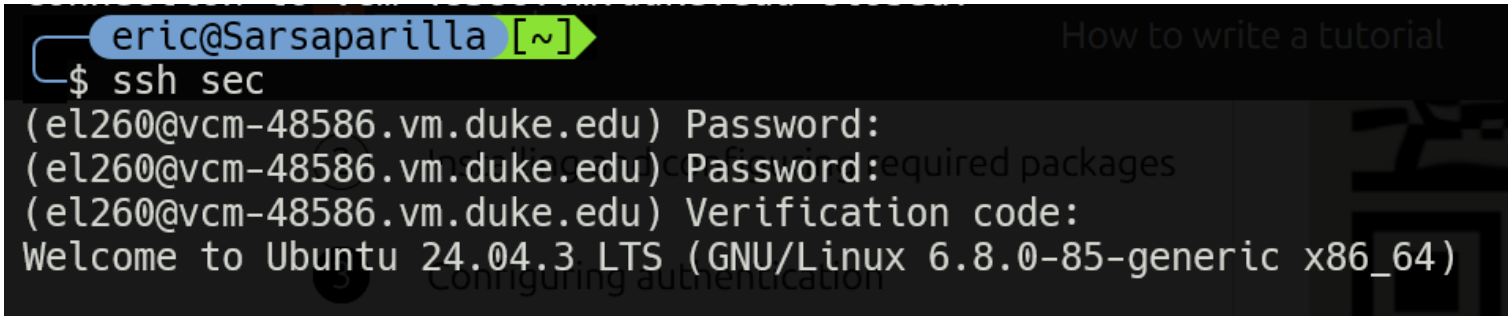
On your Linux VM, you are going to add some extra protections to the SSH server by adding Multi-Factor Authentication (MFA) to your account.
Some research will tell you how to do this using Google Authenticator or the desktop app Authy.

Show a screenshot of your SSH login with MFA code.

researching from: https://ubuntu.com/tutorials/configure-ssh-2fa

```
$ sudo apt install libpam-google-authenticator
$ sudo echo "auth required pam_google_authenticator.so" >> /etc/pam.d/sshd
$ sudo systemctl restart ssh.service
$ sudo sed 's/\(ChallengeResponseAuthentication\) no/\1 yes' /etc/ssh/sshd_config
$ google-authenticator
```

Ta da!



When done, you may remove this MFA support if you wish.

Question 11: Hydra (Online SSH Dictionary Attack) (5 points)

Hydra is an online network-based dictionary attack tool for SSH and many other protocols. For this example, the tool will take 4 input values: username file, a password file, target list, and a service (ssh). With that information, it attempts to perform a network-based authentication to the remote machine(s) on port 22. If it successfully authenticates with the remote machine, it shows the results in the standard output with the username and passwords that worked.

For this exercise, you are going to use Hydra to perform an SSH dictionary attack against a machine specially prepared for this purpose, target.colab.duke.edu. Do not use the tool on any other targets!

Implementation note: Because I've intentionally put a guessable username/password combo on this machine, I've used the ufw software firewall (based on Linux iptables) to restrict SSH access to campus IP ranges.

On your Kali VM, do the following.

Acquire a password list

A password list is a text file of likely passwords. There are many password lists out there, and Kali Linux ships with many of them pre-installed. For our purposes, we will use the Adobe "top 100" password list, which was derived from a 2013 breach of 150 million Adobe user accounts. You can find this password list in `/usr/share/wordlists/metasploit/adobe_top100_pass.txt`

Build user list (1 point)

If an attacker is doing a general scan, they will typically use a common username list, such as one of these. If an attacker is focused on a known server or organization, they will do research using public information to create a list of likely usernames.

In this scenario, let's assume we are targeting an organization's founders, which includes Susan Hypothetical, Scott Samplesberg, and Joe Exampleface. Based on this, produce a small text file of likely usernames.

Paste your username list below.

```
sh
shy
shyp
shypo
shypot
shypoth
ss
ssa
ssam
```

ssamp
ssampl
ssample
je
jex
jexa
jexam
jexamp
jexampl
sh0
sh1
sh2
sh3
ss0
ss1
ss2
ss3
je0
je1
je2
je3
s.hypothetical
susan.hypothetical
s.samplesberg
scott.samplesberg
j.exampleface
joe.exampleface
shypothetical
susanhypothetical
ssamplesberg
scottsamplesberg
jexampleface
joeexampleface
hypotheticals
hypotheticalsusan
samplesbergs
samplesbergscott
examplefacej
examplefacejoe
susan
scott
joe
hypothetical
samplesberg
exampleface
Susan
Scott
Joe
Hypothetical

```
Samplesberg
Exampleface
```

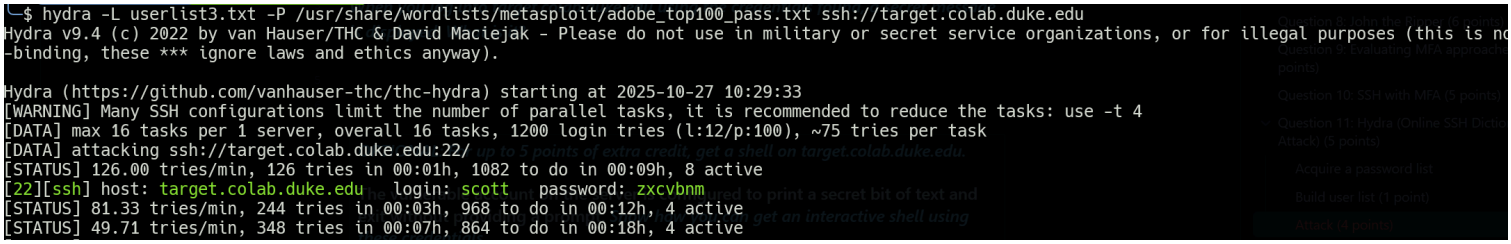
Attack (4 points)

Run hydra against `target.colab.duke.edu` using the resources gathered.

Give the hydra command you used below.

```
$ hydra -L userlist3.txt -P /usr/share/wordlists/metasploit/adobe_top100_pass.txt
ssh://target.colab.duke.edu
```

Display the successful results of your Hydra Attack below. If you don't get any results, try more (and simpler!) usernames.
Note: It is very likely that you will only be able to find one set of working credentials.



When you log into target.colab.duke.edu using the credentials found, a secret message is displayed. What is it?
"Scott Samplesberg is AWESOME!!"

OPTIONAL: For up to 5 points of extra credit, get a shell on target.colab.duke.edu.

The vulnerable account on the server is configured to print a secret bit of text and exit without providing a prompt. *Show how you can get an interactive shell using these credentials.*

(probably because the default shell is set to a program that prints and exits? maybe there's some way of changing the default shell of the user?)

```
$ ssh scott@target.colab.duke.edu /bin/bash
scott@target.colab.duke.edu's password:
ls
motd.txt
```

it's that easy???

```
cat etc/passwd
...
scott:x:1004:1004:,,,:/home/scott:/bin/bash
```

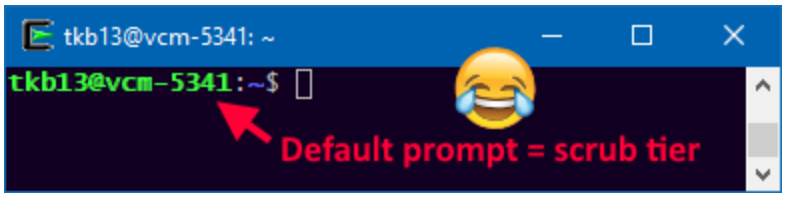
I guess so?

```
$ ls -al
total 24
dr-xr-x--- 4 scott scott  143 Oct  4 14:46 .
drwxr-xr-x 6 root  root    56 Oct  4 14:43 ..
-r-------- 1 root  root    94 Oct  4 14:46 .bash_history
-r--r--r-- 1 root  root   220 Oct  4 14:43 .bash_logout
-r-xr-xr-x 1 root  root    21 Oct  4 14:46 .bash_profile
-r--r--r-- 1 root  root  3771 Oct  4 14:43 .bashrc
dr-x------ 2 root  root    34 Oct  4 14:44 .cache
dr-xr-xr-x 3 root  root    19 Oct  4 14:45 .local
-r--r--r-- 1 root  root  2837 Oct  4 14:45 motd.txt
-r--r--r-- 1 root  root   807 Oct  4 14:43 .profile
$ cat .bash_profile
cat ~/motd.txt
exit
```

ok that makes sense! I suppose if you wanted to be more secure you could set the shell to a program that just prints text?

Question 12: Craft your prompt (3 points)

A rite of passage of UNIX users is the customization of the bash prompt. The default, even on modern Ubuntu, is pretty lame.



Using your understanding of shell color codes, develop a custom prompt by modifying the PS1 environment variable. You'll need to mark the escape codes for the shell using [and] indicators (this the shell needs to know what characters are printed vs. interpreted for cursor control purposes); details are here.

Edit the file .bashrc (sourced by the bash shell every time it is launched) in order to apply your prompt permanently.

Paste your PS1 environment variable command from .bashrc & a screenshot of the resulting prompt.

Used PROMPT_COMMAND here so that I could have more flexibility. Find the whole thing on my github
It displays the current git branch, and nonzero exit codes. Might add digital clock and python env stuff?

```
GREEN='\[\033[92;40m\]'
BLACK_GREEN='\[\033[30;102m\]'
BLUE_GREEN='\[\033[94;102m\]'
BLUE='\[\033[94;40m\]'
GREEN_BLUE='\[\033[92;104m\]'
BLACK_BLUE='\[\033[30;104m\]'
GREEN_YELLOW='\[\033[92;103m\]'
BLUE_YELLOW='\[\033[94;103m\]'
BLACK_YELLOW='\[\033[30;103m\]'
YELLOW='\[\033[93;40m\]'
RED='\[\033[91;40m\]'
BLACK_RED='\[\033[30;101m\]'
BLUE_RED='\[\033[94;101m\]'
RESET='\[\033[00m\]'

# Nerd font powerline symbols
LEFT_CIRCLE='\356\202\266'
RIGHT_CIRCLE='\356\202\264'
RIGHT_TRIANGLE='\356\202\260'
TRAPEZOIDS='\356\203\224'
PIXELATED='\356\203\206'
BRANCH_SYMBOL='\356\202\240'
LOWER_RIGHT_TRI='\356\202\272'
UPPER_LEFT_TRI='\356\202\274'
```

```
HONEYCOMB='\356\203\214'

# Box drawing symbols:
DOWN_RIGHT='\342\225\255'
UP_RIGHT='\342\225\260'
LIGHT_HORIZ='\342\224\200'

# Misc
ERROR_SYMBOL='\356\231\224'
FANCY_DOLLAR='\357\205\225'

build_prompt () {
    if [ "$color_prompt" = yes ]; then
        BASE_PS1="${debian_chroot:+
($debian_chroot)}${BLACK_BLUE}\u@\h${BLUE_GREEN}${RIGHT_CIRCLE}${BLACK_GREEN}
[\w]${RESET}"

        if [[ $1 -ne 0 ]]; then
            ERROR_STATUS="${RED}${LEFT_CIRCLE}${BLACK_RED}${ERROR_SYMBOL}
$1${BLUE_RED}${LEFT_CIRCLE}${RESET}"
        else
            ERROR_STATUS="${BLUE}${LEFT_CIRCLE}"
        fi

        if [[ $(__git_ps1) ]]; then

GIT_BRANCH="${GREEN_YELLOW}${RIGHT_TRIANGLE}${BLACK_YELLOW}${BRANCH_SYMBOL}\$(__git_ps1
'(%s)')${YELLOW}${UPPER_LEFT_TRI}${RESET}"
        else
            GIT_BRANCH="${GREEN}${RIGHT_TRIANGLE}${RESET}"
        fi

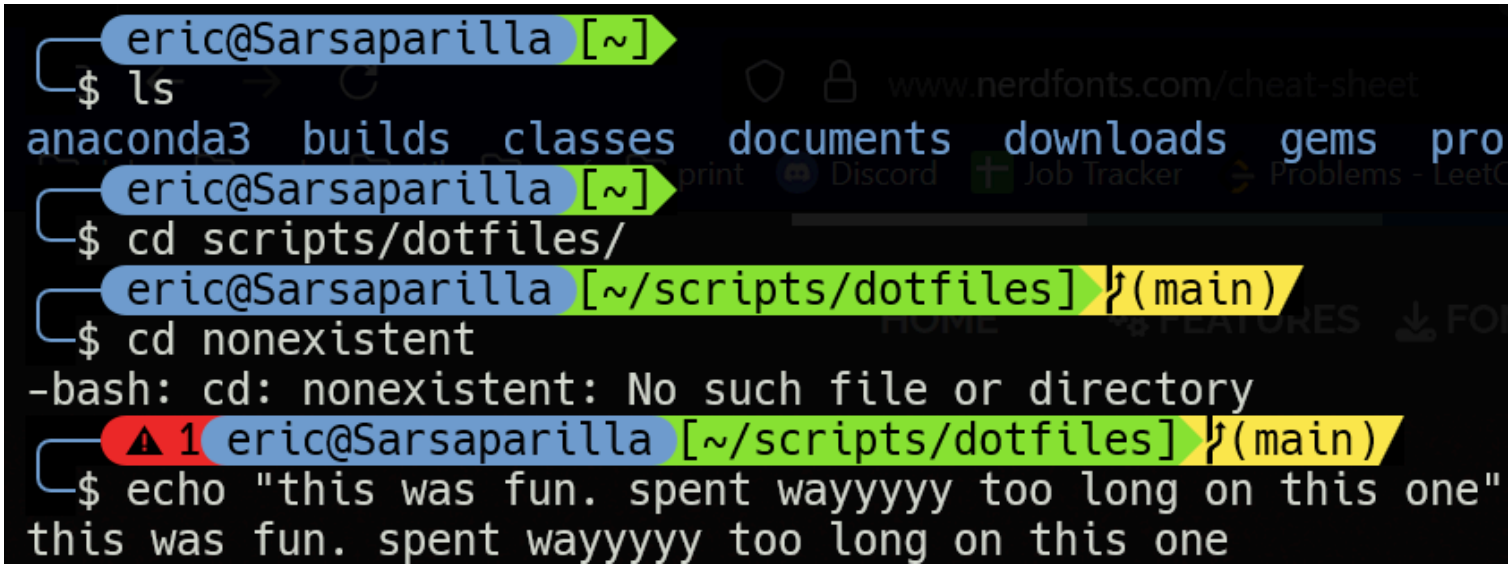
        ARROW_TOP="${BLUE}${DOWN_RIGHT}${LIGHT_HORIZ}${LIGHT_HORIZ}"
        ARROW_BOT="${BLUE}${UP_RIGHT}${LIGHT_HORIZ}${RESET}"
        echo $"${ARROW_TOP}${ERROR_STATUS}${BASE_PS1}${GIT_BRANCH}\n${ARROW_BOT}$ "
    else
        echo '${debian_chroot:+($debian_chroot)}\u@\h:\w $(parse_git_branch)$ '
    fi
    unset color_prompt force_color_prompt

    # If this is an xterm set the title to user@host:dir
    case "$TERM" in
    xterm*|rxvt*)
        echo "\[\e]0;${debian_chroot:+($debian_chroot)}\u@\h: \w\a\]$PS1"
        ;;
    *)
        ;;
    esac

    return $1
```

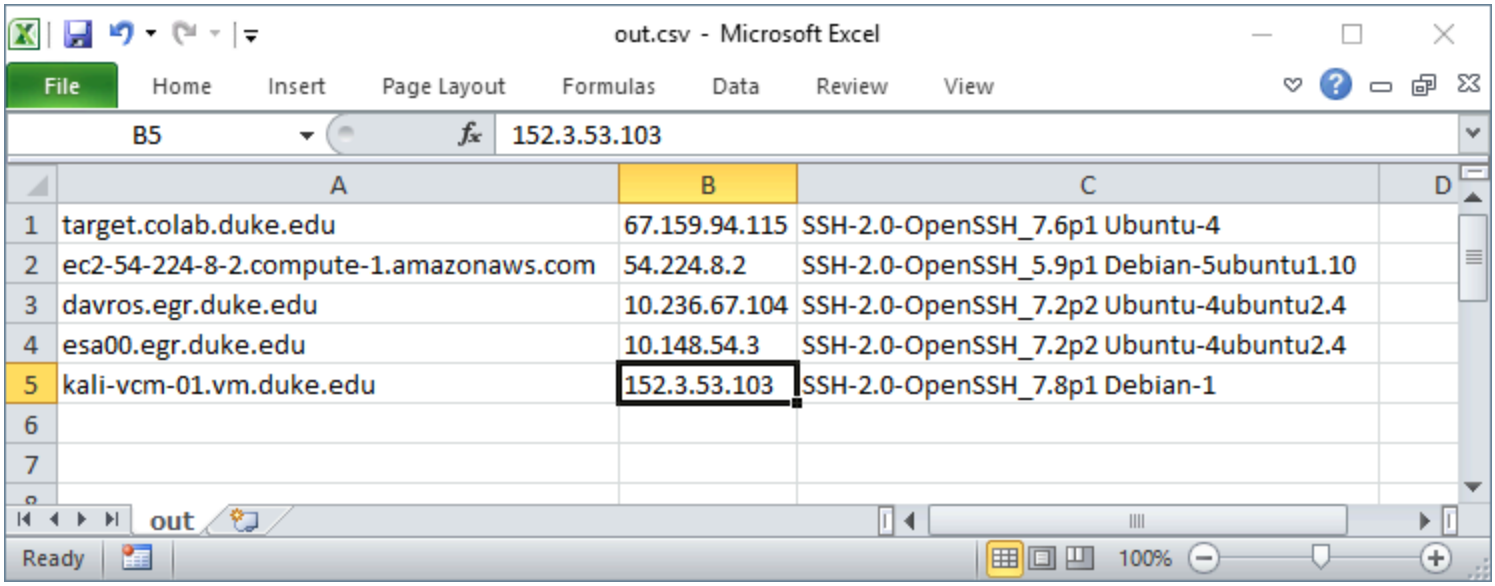
```
}

function prompt_command {
    RET=$?
    export PS1=$(build_prompt $RET)
}
PROMPT_DIRTRIM=3
export PROMPT_COMMAND=prompt_command{{}}
```



Question 13: Attack recon (6 points)

One of the first things an attacker will do when focusing on a particular target organization is conduct reconnaissance. Using your choice of tools we've learned, develop a command or script that will produce a CSV file containing the hostname, IP address, and SSH version of all the hosts listed in an input file. Example output (once loaded into Excel):



Apply this to the following hosts:

```
target.colab.duke.edu
95.179.232.162
139.84.235.222
45.77.44.223
65.20.113.226
```

(Do not scan machines other than the above. All of the above are machines administered by me either at Duke or at a cloud hosting provider.)

Your goal is to write a tiny script as quickly and easily as possible. You may include code from the internet with citation.

Provide:
(1) the command or script as copyable text (so we can try it out)
(2) a screenshot of your command plus its output

using the banner-grabbing script from homework 1, I saw that this gave the ssh identification string desired.

```
$ echo q | nc <target> 22
```

wrote a function to perform this lookup for a single target: `recon.sh` :

```
# recon.sh
# first figure out if we got the hostname or the targetip address
# pipe grep into dev null to keep it from writing an extra line to terminal
if echo $1 | grep -P "(\d+\.){3}\d+" > /dev/null ; then
    targetip=$1
    hostname=$(dig -x $1 +short | tail -1)
else
    hostname=$1
    # use tail -1 to get last line of output
    targetip=$(dig $1 +short | tail -1)
fi
banner=$(echo q | nc $targetip 22 | head -1)
echo ${hostname},${targetip},${banner}
```

went to stack overflow to figure out how to call this for every single line in a text file:

```
el260@vcm-48716 [~/hw3/q13-recon]
$ cat targets.txt | xargs -L 1 ./recon.sh >> targets.csv
el260@vcm-48716 [~/hw3/q13-recon]
$ cat targets.csv
target.colab.duke.edu,152.3.53.82,SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.14
95.179.232.162.vultrusercontent.com.,95.179.232.162,SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.13
139.84.235.222.vultrusercontent.com.,139.84.235.222,SSH-2.0-OpenSSH_9.9 FreeBSD-20250219
45.77.44.223.vultrusercontent.com.,45.77.44.223,SSH-2.0-OpenSSH_8.4p1 Debian-5+deb11u5
65.20.113.226.vultrusercontent.com.,65.20.113.226,SSH-2.0-OpenSSH_8.7
```

and then confirmation that this works in excel:

```
eric@Sarsaparilla [~/downloads]
$ wslview targets.csv
grep: /proc/sys/fs/binfmt_misc/WSLInte
WSL Interopability is disabled. Please
grep: /proc/sys/fs/binfmt_misc/WSLInte
[error] WSL Interoperability is disabl
```

	A	B	C	D	E	F
1	target.cola	152.3.53.8	SSH-2.0-OpenSSH_9.6p1	Ubuntu-3ubuntu13.14		
2	95.179.23	95.179.23	SSH-2.0-OpenSSH_8.9p1	Ubuntu-3ubuntu0.13		
3	139.84.23	139.84.23	SSH-2.0-OpenSSH_9.9	FreeBSD-20250219		
4	45.77.44.2	45.77.44.2	SSH-2.0-OpenSSH_8.4p1	Debian-5+deb11u5		
5	65.20.113.	65.20.113.	SSH-2.0-OpenSSH_8.7			

(wsl is jank in many ways, but being able to "click" on a file on my linux mount feels pretty cool)

Lastly, using your tool, identify:

Which of the IP addresses is running the earliest version of Ubuntu Linux?

Seems that only two of these addresses are running ubuntu (I am assuming the distro on version 8.7 is not running ubuntu otherwise it would show up in the banner) `95.179.232.162` is running the earlier version of openssh: 8.9p1.

A google search shows that this version of openssh was present on ubuntu launchpadfor release Jammy Jellyfish, whereas the other one was present for Noble Numbat.

What version of Ubuntu Linux is it running? Note: I am not asking for the SSH version found above, but rather the overall OS version (18.04, 18.10, 19.04, etc.). You will need to research the SSH version string to identify this. Note: It may be helpful to learn the relationship between Ubuntu codenames (e.g. “Lunar Lobster”) and release version numbers (e.g. “23.04”).

A quick search shows that Jammy Jellyfish is Ubuntu version 22.04

Question 14: Keeping up with the news (10 points)

One absolutely essential attribute to the field of information security is that it changes. While most of the base theory we're talking about will remain true long into the future, attackers and defenders are locked into a continuous arms race of ever increasing sophistication. As such, a core skill for any security practitioner is to keep up to date with developments in the field.

Below are five IT security news sources:

- Security Week
- ThreatPost
- Reddit /r/netsec
- The Record
- Risky Business News

Browse these, and pick out an article from the last 30 days related to one of the following topics: networking, cryptography, user authentication, malware, or denial of service attacks.

Based on this article, ***do the following:***

- Provide a brief summary, including a link to the article. (3)
- Identify which aspects of the CIA triad are at play and how. (1)
- Explain the threat model(s) at play (assets at risk, vulnerability at play, attacker's capabilities and knowledge). (2)
- If your article is explaining an attack, describe a defense that would have mitigated the attack and explain why it would be effective. Use the threat model in your analysis.
  -or-
  If your article is explaining a defense, describe an attack that the defense would mitigate and explain why it would be effective. Use the threat model in your analysis. (4)

Question 15: Manipulating binary file formats (5 points)

Question 0 of this assignment involved detecting and decoding a polyglot PDF that was also a valid JPEG. ***To receive credit for this question, the answers PDF you submit must also be a polyglot, but rather than a PDF+JPEG polyglot, it should be a PDF+ZIP polyglot. The ZIP aspect should contain (1) your public SSH key (the one submitted earlier to Encrypted Thing Giver in homework 2) and (2) a picture of a fat dog (<100kB).***

The construction of a PDF+ZIP polyglot is easier than you may guess because of the peculiar format of ZIP files. You may consult the ZIP file format article on Wikipedia, the napkin drawings by Julia Wolf in PoC||GTFO 01:05 (“This ZIP is also a PDF”), or this way-way-too-detailed presentation deck also by Julia Wolf.

HINTS: Does a zip file have a header? Where is its central directory?